OopsMate

Swoyam Pokharel
Prakriti Regmi ~ Siman Giri

Date created: 2026-02-26
Date modified: 2026-02-26

Contents

1. Project Overview 1
2. Work Completed So Far 2
   2.1. Background Research 2
   2.2. Requirement Gathering 6
   2.3. Design 6
   2.4. Implementation 7
      2.4.1. Development Environment 7
      2.4.2. Implemented Features 7
      2.4.3. Testing 8
      2.4.4. Tools Used 9
   2.5. Progress Showcase 9
      2.5.1. Performance (Search Speed) 9
      2.5.2. Rating Estimate 10
3. Challenges Faced 12
4. Remaining Work 12
   4.1. Context 12
   4.2. Phase 1: AVX512 and VNNI Support for nnuebie (March 2026) 13
   4.3. Phase 2: nnuebie Integration into OopsMate (By April 10, 2026) 13
   4.4. Phase 3: Final Development (April 2026) 13
   4.5. Phase 4: Deliverables and Presentation (May 2026) 13
5. Meeting Summary 13
6. Self Assessment 14

1. Project Overview

OopsMate is a chess engine developed in Rust with the primary objective of benchmarking various techniques employed in modern chess programming to quantify their impact on playing strength. The project serves dual purposes: advancing understanding of algorithmic improvements in game-playing systems and providing a modular framework for evaluating chess engine components.

The motivation for this project stems from two factors. First, a longstanding personal interest in understanding the internal workings of chess engines and the underlying computational principles. Second, the desire to acquire proficiency in Rust programming through a substantial, technically challenging project that demands high performance and precise memory management.

Objectives:

- Achieve a playing strength of at least 1800 Elo on the Computer Chess Rating List (CCRL) scale.
- Systematically benchmark and rank search and evaluation techniques based on their contribution to playing strength improvement.

Deliverables:

- A crate named `nnuebie` that implements Stockfish's NNUE (Efficiently Updatable Neural Network) evaluation function using the HalfKA_hm_v2 architecture in pure Rust. This library provides an API enabling integration with other chess engines.
  - This crate addresses a gap in the Rust ecosystem, as existing implementations such as those by VedantJoshi1409 (2023) and dshawul (2018) are not available in Rust.
- A high-performance crate named `strikes` that provides pre-computed attack tables and path utilities for efficient chess piece movement calculation.
- The complete engine compiled as a single binary, integrating all supporting crates.

2. Work Completed So Far

2.1. Background Research

A comprehensive literature review was conducted, examining 18 sources comprising academic papers, technical reports, and documentation from existing engine implementations. The review encompassed foundational works in chess programming, including Shannon's (1950) seminal paper on computer chess, as well as contemporary approaches such as neural network-based evaluation functions. The reviewed systems included Stockfish (representing the pinnacle of classical engine development), AlphaZero, and LC0 (representing learning-based approaches via self-play).

```bibtex
@article{shannon_1950_programming,
    author = {Shannon, Claude E.},
    month = {03},
    pages = {256-275},
    title = {Programming a Computer for Playing Chess},
    doi = {10.1080/14786445008521796},
    url = {https://vision.unipv.it/IA1/aa2009-2010/ProgrammingaComputerforPlayingChess.pdf},
    volume = {41},
    year = {1950},
    journal = {The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science}
}

@article{marsland,
    author = {Björnsson, Y. and Marsland, T.A.},
```

```bibtex
    month = {01},
    pages = {23-41},
    title = {A Review Of Game-Tree Pruning},
    doi = {10.1016/s0020-0255(99)00097-3},
    url = {https://webdocs.cs.ualberta.ca/~tony/OldPapers/icca.1986-03.3-18.pdf},
    volume = {122},
    year = {2000},
    journal = {Information Sciences}
}

@article{knuth_1975_an,
    author = {Knuth, Donald E. and Moore, Ronald W.},
    pages = {293-326},
    title = {An Analysis of alpha-beta Pruning},
    doi = {10.1016/0004-3702(75)90019-3},
    url = {https://kodu.ut.ee/~ahto/eio/2011.07.11/ab.pdf},
    volume = {6},
    year = {1975},
    journal = {Artificial Intelligence}
}

@thesis{bijl_2021_exploring,
    author = {Bijl, Pieter and Tiet, Anh and Bal, H E},
    month = {07},
    title = {Exploring Modern Chess Engine Architectures},
    url = {https://www.cs.vu.nl/~wanf/theses/bijl-tiet-bscthesis.pdf},
    year = {2021}
}

@thesis{hash_funtions,
    author = {Niklas Fiekas},
    month = {apr},
    title = {Finding Hash Functions for Bitboard Based Move Generation},
    url = {https://backscattering.de/magics2.pdf},
    year = {2018}
}


@thesis{Brange,
    author    = {Henrik Brange},
    title     = {Evaluating Heuristic and Algorithmic Improvements for Alpha-Beta Search in a Chess Engine},
    year      = {2021},
    url       = {https://lup.lub.lu.se/luur/download?func=downloadFile&recordOId=9069249&fileOId=9069251},
    school    = {Lund University},
    note      = {ISSN 1651-2197, LU-CS/HBG-EX: 2021}
}

@thesis{tessaract,
    author = {Sander Vrzina},
    title = {Piece By Piece Building a Strong Chess Engine.},
```

```bibtex
    url = {https://www.cs.vu.nl/~wanf/theses/vrzina-bscthesis.pdf},
    month = {07},
    year = {2023}
}

@thesis{sophie,
    author = {Columbia, Sophie and Mohan, Raghuveer and Klima, Vicky and Mohan, },
    title = {Chess Move Generation Using Bitboards},
    url = {https://libres.uncg.edu/ir/asu/f/Columbia_Sophie_Spring%202023_thesis.pdf},
    year = {2023}
}


@misc{parallel_chess_searching,
    author = {Rasmussen, David},
    title = {Parallel Chess Searching and Bitboards},
    url = {https://www.cs.cmu.edu/afs/cs/academic/class/15418-s12/www/competition/www.contrib.andrew.cmu.edu/~jvirdo/rasmussen-2004.pdf},
    year = {2004}
}

@thesis{alphadeepchess,
    title        = {AlphaDeepChess: motor de ajedrez basado en podas alpha-beta = AlphaDeepChess: chess engine based on alpha-beta pruning},
    author       = {Juan Girón Herranz and Yi Wang Qiu},
    year         = {2025},
    school       = {Universidad Complutense de Madrid},
    type         = {Trabajo de Fin de Grado},
    institution  = {Facultad de Informática},
    address      = {Madrid, España},
    month        = {junio},
    url = {https://docta.ucm.es/rest/api/core/bitstreams/4e289e34-0b84-4c1b-9d19-bc0911cfb48b/content},
}


@misc{mastering,
    author = {Silver, David and Hubert, Thomas and Schrittwieser, Julian and Antonoglou, Ioannis and Lai, Matthew and Guez, Arthur and Lanctot, Marc and Sifre, Laurent and Kumaran, Dharshan and Graepel, Thore and Lillicrap, Timothy and Simonyan, Karen and Hassabis, Demis},
    month = {12},
    title = {Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm},
    url = {https://arxiv.org/pdf/1712.01815},
    year = {2017}
}

@article{mcts,
```

```bibtex
    title={Monte Carlo Tree Search: A Review of Recent Modifications and Applications},
    author={Świechowski, Maciej and Godlewski, Konrad and Sawicki, Bartosz and Mańdziuk, Jacek},
    journal={arXiv preprint arXiv:2103.04931},
    year={2022},
    month={jun},
    day={11},
    url={https://arxiv.org/pdf/2103.04931}
}

@article{zobrist,
    title={A New Hashing Method With Application For Game Playing},
    author={Albert L. Zobrist},
    journal={The University Of Wisconsin},
    year={1970},
    month={apr},
    url={https://research.cs.wisc.edu/techreports/1970/TR88.pdf}
}

@book{deGroot1965,
    author    = {Adriaan D. de Groot},
    title     = {Thought and Choice in Chess},
    publisher = {Mouton},
    address   = {The Hague},
    year      = {1965},
    note      = {2nd edition published 1978},
    url       = {https://iiif.library.cmu.edu/file/Simon_box00082_fld06608_bdl0002_doc0002/Simon_box00082_fld06608_bdl0002_doc0002.pdf},
}


@techreport{negascout,
    title       = "A Minimax Algorithm faster than NegaScout",
    author      = "Aske Plaat",
    institution = "Erasmus University Rotterdam",
    year        = 1997,
    month       = jan,
    url = {https://arxiv.org/pdf/1404.1511}

}


@misc{Nasu2018NNUE,
    author       = {Yu Nasu},
    title        = {NNUE: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi}, translator   = {Dominik Klein},
    year         = {2018},
    url = {https://github.com/asdfjkl/nnue/blob/main/nnue_en.pdf},
    note         = {Unofficial English translation of the original Japanese paper.},
}

```

```bibtex
@misc{Stockfish2025NNUEWiki,
    author       = {Stockfish-Team},
    title        = {NNUE: Efficiently Updatable Neural Network — Stockfish Docs},
    year         = {2025},
    url = {https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html},
}

@misc{Nasu2018NNUEOriginal,
    author       = {Yu Nasu},
    title        = {NNUE: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi},
    year         = {2018},
    url          = {https://www.apply.computer-shogi.org/wcsc28/appeal/the_end_of_genesis_T.N.K.evolution_turbo_type_D/nnue.pdf},
    note         = {Original Japanese paper. English translation available at https://github.com/asdfjkl/nnue/blob/main/nnue_en.pdf},
    howpublished = {Presented at WCSC28}
}
```

2.2. Requirement Gathering

The requirements for this project were established through an iterative process. Initially, a foundational chess engine was required that could correctly play legal chess moves and evaluate positions. This foundational requirement was subsequently expanded to include benchmarking capabilities, enabling systematic comparison between successive versions of the engine. The development methodology adopted a version-based approach, where each significant algorithmic enhancement was implemented as a separate version, facilitating controlled experimentation and performance measurement.

The project scope was refined to include three primary deliverables: the main engine binary, the `strikes` crate for move generation utilities, and the `nnuebie` crate for neural network-based evaluation. This modular architecture enables independent development and testing of components while maintaining integration flexibility.

2.3. Design

Unlike conventional software projects that require database schemas or user interface wireframes, chess engine development focuses on algorithmic architecture and data structure design. The codebase was structured into modular components to facilitate maintainability and testing:

- Core Position Representation: The `position` module handles game state management, including piece placement, turn management, castling rights, and en passant squares.
- Move Generation (`strikes` crate): A dedicated crate providing pre-computed attack tables using magic bitboard techniques for efficient sliding piece move generation.
- Evaluation (`evaluate` module): Implements position evaluation using piece-square tables with tapered evaluation for smooth transition between game phases.
- Search Algorithm (`search` module): Contains alpha-beta search with iterative deepening, various pruning techniques, and move ordering heuristics.

- Neural Network Evaluation (`nnuebie` crate): Work in progress crate implementing Stockfish's NNUE architecture for advanced position evaluation.

The design prioritizes performance through aggressive compiler optimizations, including link-time optimization (LTO), single codegen unit compilation, and fat LTO settings in release builds.

2.4. Implementation

2.4.1. Development Environment

- Rust toolchain (Stable channel, version 1.93.0)

```
cargo 1.93.0 (083ac5135 2025-12-15)
```

- Git for version control with GitHub hosting
- Neovim text editor with Tmux terminal multiplexer for development workflow
- Additional tools: clippy for linting, rust_analyzer Language Server Protocol (LSP) implementation, and perf for performance profiling

2.4.2. Implemented Features

The following search and evaluation techniques have been implemented and benchmarked across 16 released versions:

Search Optimizations:

- Alpha-Beta Pruning: Fundamental minimax enhancement that eliminates irrelevant branches in the game tree.
- Iterative Deepening: Repeatedly searches to increasing depths, improving move ordering and enabling time management.
- Transposition Table with Zobrist Hashing: Enables detection of previously evaluated positions, avoiding redundant search.
- Quiescence Search: Extends search to capture sequences, preventing horizon effects from obscuring tactical positions.
- Principal Variation Search (PVS): Optimizes search by assuming a best move exists and exploiting this assumption.
- Null Move Pruning: Exploits the intuition that having an extra move provides positional advantage.
- Late Move Reductions (LMR): Reduces search depth for moves likely to be poor, focusing computation on promising lines.
- Futility Pruning: Prunes moves that cannot improve the search result based on positional evaluation.
- Reverse Futility Pruning: Prunes positions that are clearly winning, allowing deeper search in winning branches.
- Razoring: Reduces search depth when a position appears worse than a shallow search threshold.
- Check Extension: Extends search depth when checking moves are found, improving tactical accuracy.
- Aspiration Windows: Narrows alpha-beta bounds based on previous search results, improving efficiency.

- Internal Iterative Deepening (IID): Performs shallow searches to improve move ordering in principal variation nodes.
- ProbCut: A probabilistic enhancement to futility pruning based on shallow search results.
- Singular Extensions: Extends moves that are significantly better than alternatives.
- Dynamic Null Move Pruning: Adaptive null move pruning based on remaining depth and position characteristics.
- Lazy SMP (Symmetric Multi-Processing): Parallel search using multiple threads with shared transposition table.
- Internal Mate Detection: Recognition of checkmate patterns during search.

Evaluation Enhancements:

- Tapered Evaluation: Smooth interpolation between middlegame and endgame evaluation values.
- Piece-Square Tables (PeSTO Values): Position-dependent evaluation using empirically tuned piece-square tables.
- Static Exchange Evaluation (SEE): Evaluates the material outcome of capture sequences without performing full search.
- History Heuristic: Records move history to improve move ordering based on previous search successes.
- Killer Move Heuristic: Preserves good capture and check moves for potential reuse.
- Transposition Table Move Ordering: Prioritizes moves that have previously produced good results.

Move Generation:

- Magic Bitboards: Efficient sliding piece attack calculation using pre-computed tables.
- MVV-LVA (Most Valuable Victim - Least Valuable Attacker): Captures ordering based on material trade calculations.
- Make/Unmake Architecture: Efficient position update mechanism enabling full move history.

Interface:

- UCI (Universal Chess Interface): Standard protocol for communication with chess GUIs.

2.4.3. Testing

Testing methodology employed multiple complementary approaches:

1. PERFT (Performance Test): Validation of move generation correctness by comparing computed node counts against known values at various depths. This ensures fundamental move generation accuracy.
2. Position Validation: Testing against established test suites including:
    - Start Position (standard opening)
    - KiwiPete (tactical positions)
    - Giuoco Piano (middlegame structure)
    - Rook+Pawn endgame scenarios
    - WAC-2 (Win at Chess) tactical puzzles
3. Self-Play Testing: Each new version is tested against previous versions to measure Elo improvement.
4. Engine vs. Engine Matching: Competitive testing against established engines including:
    - Stockfish with UCI Elo limiting (calibrated to FIDE scale)

    - Sayuri (1807 CCRL rated)
    - Goldfish (reference engine)
5. Performance Profiling: Using `perf` to identify CPU hotspots and optimize critical code paths.
6. Manual Verification: Position-by-position analysis using chess databases and theoretical knowledge.

2.4.4. Tools Used

- Programming Language: Rust (no external dependencies)
- Project Management: Trello
- Development Tools:
    - Neovim (text editor)
    - Tmux (terminal multiplexer)
    - Clippy (linting)
    - Rust Analyzer (LSP)
    - Perf (Linux profiling)
    - Git (version control)
- Testing Infrastructure:
    - CuteChess (GUI wrapper for engine communication)
    - CuteChess-CLI (automated match execution)
    - Ordo (Elo rating calculation with anchored ratings)
    - Modern.pgn (opening book for standardized test positions)

2.5. Progress Showcase

- GitHub Releases (16 versions): https://github.com/PS-Wizard/OopsMate/releases

2.5.1. Performance (Search Speed)

The benchmark results demonstrate substantial search performance across various positions:

```
// Summary (Single-Threaded Metrics)
Total Nodes: 2,556,134
Total Time:  0.359s
Overall NPS: 7,120,150

// Multi-Threaded Performance:
1 Thread: 0.910s
4 Threads: 0.376s
Speedup: 2.42x
```

Selected benchmark results from standard test positions:

Start Position (Depth 14):

- Best Move: g1f3
- Score: 38 centipawns
- Nodes: 204,707
- NPS: 7,873,346

KiwiPete (Depth 14):

- Best Move: e2a6

- Score: −22 centipawns
- Nodes: 917,912
- NPS: 6,700,087

Middlegame - Giuoco Piano (Depth 14):

- Best Move: c4d5
- Score: −10 centipawns
- Nodes: 1,139,405
- NPS: 6,742,041

Endgame - Rook+Pawn (Depth 16):

- Best Move: c2c6
- Score: 0 centipawns
- Nodes: 180,017
- NPS: 15,001,416

Tactical - WAC-2 (Depth 13):

- Best Move: g3d6
- Score: 110 centipawns
- Nodes: 114,093
- NPS: 7,606,200

Multi-Threaded Depth 18 (KiwiPete):

- Nodes: 5,566,454
- NPS: 6,123,711

2.5.2. Rating Estimate

The engine's playing strength has been evaluated through competitive matches against established engines:

Against Stockfish (2100 UCI Elo limited): Positive result indicating strength above this threshold.

```
Score of OopsMate-v16 vs SF-2100: 45 - 37 - 18  [0.540] 100
OopsMate-v16 playing White: 24 - 18 - 8  [0.560] 50
OopsMate-v16 playing Black: 21 - 19 - 10  [0.520] 50
Elo difference: +27.9 +/- 62.4, LOS: 81.2 %, DrawRatio: 18.0 %
```

Against Sayuri (1807 CCRL Rated): Achieves parity with established engine.

```
Score of OopsMate-v16 vs Sayuri 2018.05.23: 99 - 97 - 104  [0.503] 300
OopsMate-v16 playing White: 54 - 40 - 56  [0.547] 150
OopsMate-v16 playing Black: 45 - 57 - 48  [0.460] 150
Elo difference: +2.3 +/- 31.8, LOS: 55.7 %, DrawRatio: 34.7 %
```

Note: Stockfish's UCI Elo Limit option is calibrated to reflect human FIDE scale ratings, whereas CCRL ratings are derived from engine-versus-engine competition, representing a different Elo scale.

Testing methodology:

- Initial validation via gauntlet matches against multiple engines (Goldfish, Sayuri, Stockfish at various Elo levels)
- Incremental testing: each new version tested against the previous version
- Progressive Elo threshold testing with Stockfish UCI limiting

- Recent testing against CCRL-rated engines for calibrated comparisons
- Time control: 5+0.05 (5 seconds base + 0.05 seconds increment)
- Match rounds: 100-300 games per test
- Opening book: Modern.pgn for first 16 plies

Reference commands for testing infrastructure:

```
// Gauntlet match against multiple engines
cutechess-cli \
  -engine cmd=./oops_mate name="OopsMate" \
  -engine cmd=stockfish name="SF-1900" option."UCI_LimitStrength"=true option."UCI_Elo"=1900 option."Hash"=64 \
  -engine cmd=./goldfish name="Goldfish" option."Hash"=64 \
  -engine cmd=./sayuri name="Sayuri" option."Hash"=64 \
  -each proto=uci tc=5+0.05 \
  -rounds 100 \
  -openings file=Modern.pgn format=pgn order=random plies=16 \
  -pgnout gauntlet.pgn \
  -concurrency 8 \
  -tournament gauntlet

// Match against Sayuri
cutechess-cli \
  -engine cmd=./oops_mate \
  -engine cmd=./sayuri \
  -each proto=uci tc=5+0.05 \
  -rounds 300 \
  -recover \
  -wait 1 \
  -openings file=Modern.pgn format=pgn order=random plies=16 \
  -pgnout games.pgn \
  -concurrency 8 \
  -draw movenumber=40 movecount=8 score=10 \
  -resign movecount=5 score=600

// Match against previous release
cutechess-cli \
  -engine cmd=./oops_mate name="OopsMate-v11"\
  -engine cmd=./oops_mate_v10_aspiration_windows name="OopsMate-v10" \
  -each proto=uci tc=5+0.05 \
  -rounds 300 \
  -recover \
  -wait 1 \
  -openings file=Modern.pgn format=pgn order=random plies=16 \
  -pgnout games.pgn \
  -concurrency 8 \
  -draw movenumber=40 movecount=8 score=10 \
  -resign movecount=5 score=600

// Ordo calculation for anchored rating (Sayuri = 1807 CCRL)
./ordo -a 1807 -A "Sayuri" -p <oops_vs_sayuri.pgn>
```

3. Challenges Faced

- Magic Bitboard Comprehension: Initial difficulty in understanding the mathematical foundations underlying magic bitboard attack table generation. The implementation was subsequently migrated to PEXT-based attack generation, simplifying the approach.
- Project Management: Maintaining task tracking proved challenging; completed tasks were sometimes not logged, and additional features were implemented beyond the original scope.
- Rust Proficiency: Despite initial concerns, Rust's borrow checker proved manageable due to the nature of chess programming using bitboards, where copying is inexpensive and aliasing patterns are well-structured.
- Algorithm Understanding: Each newly implemented technique required significant study to understand both the theoretical foundations and practical implementation considerations.
- NNUE Implementation: The most significant challenge has been understanding the neural network evaluation approach. Documentation provides insufficient detail for implementing Stockfish's NNUE evaluation function. Progress continues, as exact matching with Stockfish's evaluation has been achieved; however, the computational bottleneck introduced by the current implementation has necessitated optimization work.
- CPU Intrinsics: Working with low-level CPU instructions (AVX2, AVX512, PEXT, prefetching) presented a learning curve, as this was the first project requiring such optimizations.

4. Remaining Work

4.1. Context

Completion of the `nnuebie` crate remains the primary outstanding task. The core evaluation functionality has been implemented and produces exact numerical matches with Stockfish's evaluation. However, performance bottlenecks have been identified:

Current nnuebie Performance Metrics:

```
Loading networks...
Benchmarking 10,000,000 evaluations...
Full Refresh Throughput: 897,472.96 evaluations/sec

Benchmarking Incremental Update...
Incremental Update Throughput: 1,768,581.83 evaluations/sec
Speedup: 1.97x
```

When integrated with OopsMate, the NNUE evaluation causes significant NPS reduction (from 7 million to 500k), indicating that the NNUE implementation is the primary performance bottleneck.

Performance profiling reveals:

```
34.70%  benchmark  benchmark  [<nnuebie::layers::AffineTransform as nnuebie::layers::Layer>::propagate
17.37%  benchmark  benchmark  [.] nnuebie::nnue::NNUEProbe::evaluate
```

```
3 15.18%  benchmark  benchmark  [.]
  nnuebie::finny_tables::update_accumulator_refresh_cache
4 7.58%  benchmark  benchmark  [.]
  nnuebie::accumulator::update_accumulators_single_pass_avx2
5 5.81%  benchmark  benchmark  [.]
  nnuebie::finny_tables::update_accumulator_refresh_cache
```

The next phase involves implementing AVX512 and VNNI (Vector Neural Network Instructions) support to accelerate neural network inference. VNNI is expected to significantly improve performance as profiling indicates the forward propagation function as the primary bottleneck.

4.2. Phase 1: AVX512 and VNNI Support for nnuebie (March 2026)

- Implement AVX512 vector instructions for neural network inference
- Add VNNI acceleration for matrix multiplication operations
- Optimize accumulator refresh and propagation routines

4.3. Phase 2: nnuebie Integration into OopsMate (By April 10, 2026)

- Integrate optimized nnuebie crate into main engine
- Benchmark NNUE-enhanced version against current handcrafted evaluation
- Verify playing strength improvement

4.4. Phase 3: Final Development (April 2026)

- Codebase cleanup and documentation
- Comprehensive testing against reference engines
- Report writing and presentation preparation

4.5. Phase 4: Deliverables and Presentation (May 2026)

- Final binary delivery
- Project documentation
- Oral presentation

5. Meeting Summary

- Number of Meetings Held: 15
- Number of Feedback Points Received: 23
- Number of Actions Implemented: 21 (91% implementation rate)

Feedback categories included:

- Technical guidance on search algorithm implementation (8 points)
- Evaluation function optimization strategies (5 points)
- Project management and documentation (4 points)
- Testing methodology and benchmarking approaches (4 points)
- NNUE implementation guidance (2 points)

6. Self Assessment

- Progress: Slightly Behind Schedule
- Confidence: High - Core engine functionality is complete; remaining work focuses on NNUE optimization and integration
- The project has achieved substantial progress with 16 released versions demonstrating systematic improvement in playing strength through methodical implementation of search and evaluation techniques.

I confirm that the work reported above is my own and accurately reflects my project progress to date